Machine Learning (ML) Pipeline

1. Introduction

An **ML pipeline** is a structured sequence of steps used to develop, train, and deploy machine learning models efficiently. It ensures a smooth workflow, from raw data processing to model deployment.

2. Key Stages of an ML Pipeline

1. Data Collection

- **Sources:** Databases, APIs, Web Scraping, IoT Devices
- **Formats:** CSV, JSON, Images, Videos
- **Tools:** Pandas, BeautifulSoup, Scrapy

2. Data Preprocessing

- **Handling Missing Values** (Imputation, Deletion)
- **Feature Scaling** (Standardization, Normalization)
- **Encoding Categorical Data** (One-Hot Encoding, Label Encoding)
- **Outlier Detection & Treatment** (Z-score, IQR method)

```
from sklearn.preprocessing import StandardScaler, OneHotEncoder
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

3. Feature Engineering

- **Feature Selection** (Removing irrelevant features)
- **Feature Extraction** (PCA, t-SNE)
- **Feature Transformation** (Log transformation, Box-Cox)

```
from sklearn.decomposition import PCA
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X)
```

4. Splitting Data

- **Train-Test Split (80-20, 70-30, etc.)**
- **Cross-Validation (K-Fold, Stratified K-Fold, Leave-One-Out CV)**

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

5. Model Selection

- **Algorithms:** Decision Trees, SVM, Random Forest, XGBoost
- **Hyperparameter Tuning:** GridSearchCV, RandomizedSearchCV

```
from sklearn.ensemble import RandomForestClassifier
clf = RandomForestClassifier(n_estimators=100, max_depth=5)
clf.fit(X_train, y_train)
```

6. Model Evaluation

- **Classification Metrics:** Accuracy, Precision, Recall, F1-Score, ROC-AUC
- **Regression Metrics:** MSE, RMSE, R^2 Score

```python
from sklearn.metrics import accuracy_score
y_pred = clf.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
```

7. Model Deployment

- **Flask / FastAPI** for Web Deployment
- **Streamlit** for Interactive Apps
- **Docker & Kubernetes** for Scaling

```python
from flask import Flask, request, jsonify
app = Flask(__name__)

@app.route('/predict', methods=['POST'])
def predict():
    data = request.json
    prediction = clf.predict([data['features']])
    return jsonify({'prediction': prediction.tolist()})

if __name__ == '__main__':
    app.run(debug=True)
```

3. ML Pipeline Flowchart

📌 **Data Collection → Preprocessing → Feature Engineering → Splitting Data → Model Training → Evaluation → Deployment**

4. Interview Questions & Answers

Q1: What is an ML pipeline?

A: An ML pipeline is a sequence of steps including **data preprocessing, feature engineering, model selection, evaluation, and deployment**, ensuring an efficient machine learning workflow.

Q2: Why do we need feature scaling?

A: Feature scaling ensures that all features contribute equally, preventing algorithms from giving higher weight to large-magnitude features.

Q3: How is cross-validation different from train-test split?

A: Cross-validation splits data into multiple training and validation sets, ensuring better generalization, while train-test split evaluates performance on a single holdout set.

Q4: How do you deploy an ML model?

A: Models can be deployed using **Flask/FastAPI (Web API), Streamlit (Interactive UI), Docker (Containerization), and Kubernetes (Scaling in Production).**

---

5. Important Links for Further Learning

🔗 **Scikit-Learn Pipeline Documentation:** https://scikit-learn.org/stable/modules/compose.html

🔗 **Machine Learning Pipeline Guide by Towards Data Science:** https://towardsdatascience.com/understanding-ml-pipelines-from-data-ingestion-to-model-deployment-55cf7b2b6213

🔗 **ML Deployment with Flask:** https://towardsdatascience.com/how-to-deploy-machine-learning-models-as-a-web-application-using-flask-805f0f287d5e